Micro-programmed State Machines

Lecture 10

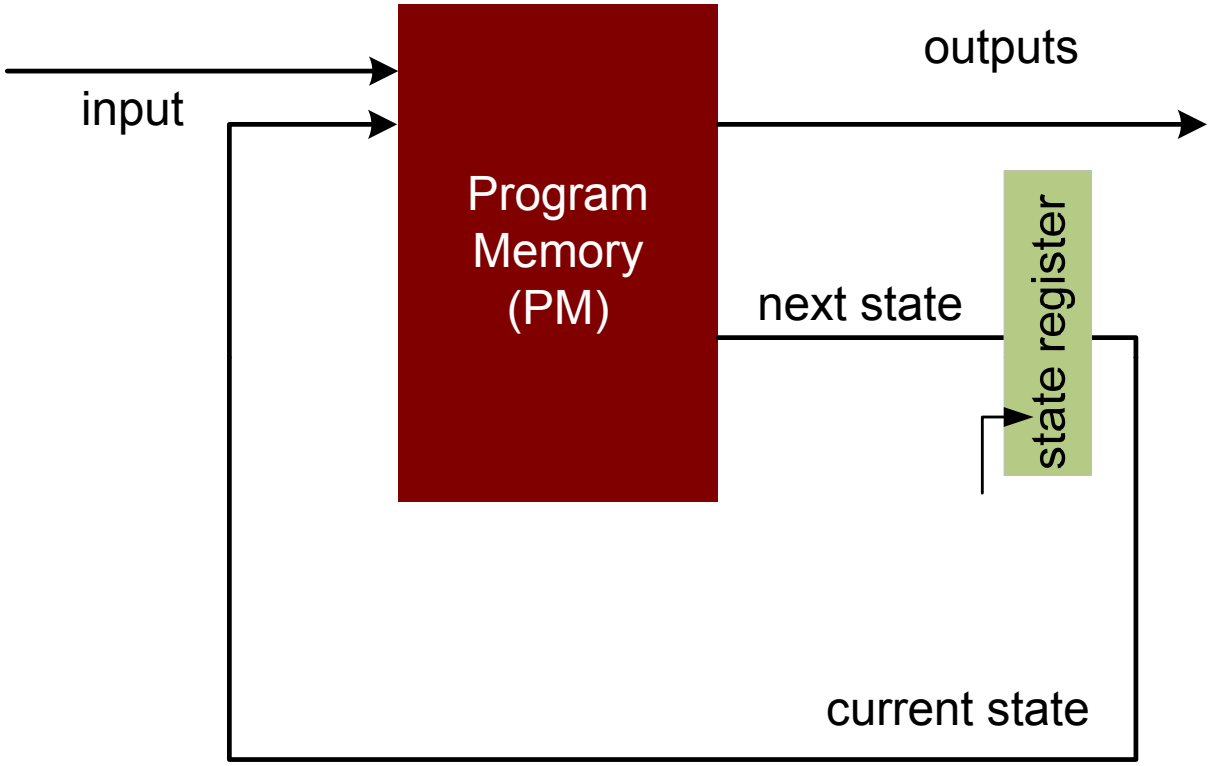
Dr. Shoab A. Khan

Micro-programmable State machines

- A Micro-Programmable State machine substitutes combinational cloud of FSM with Programmable Memory (PM)

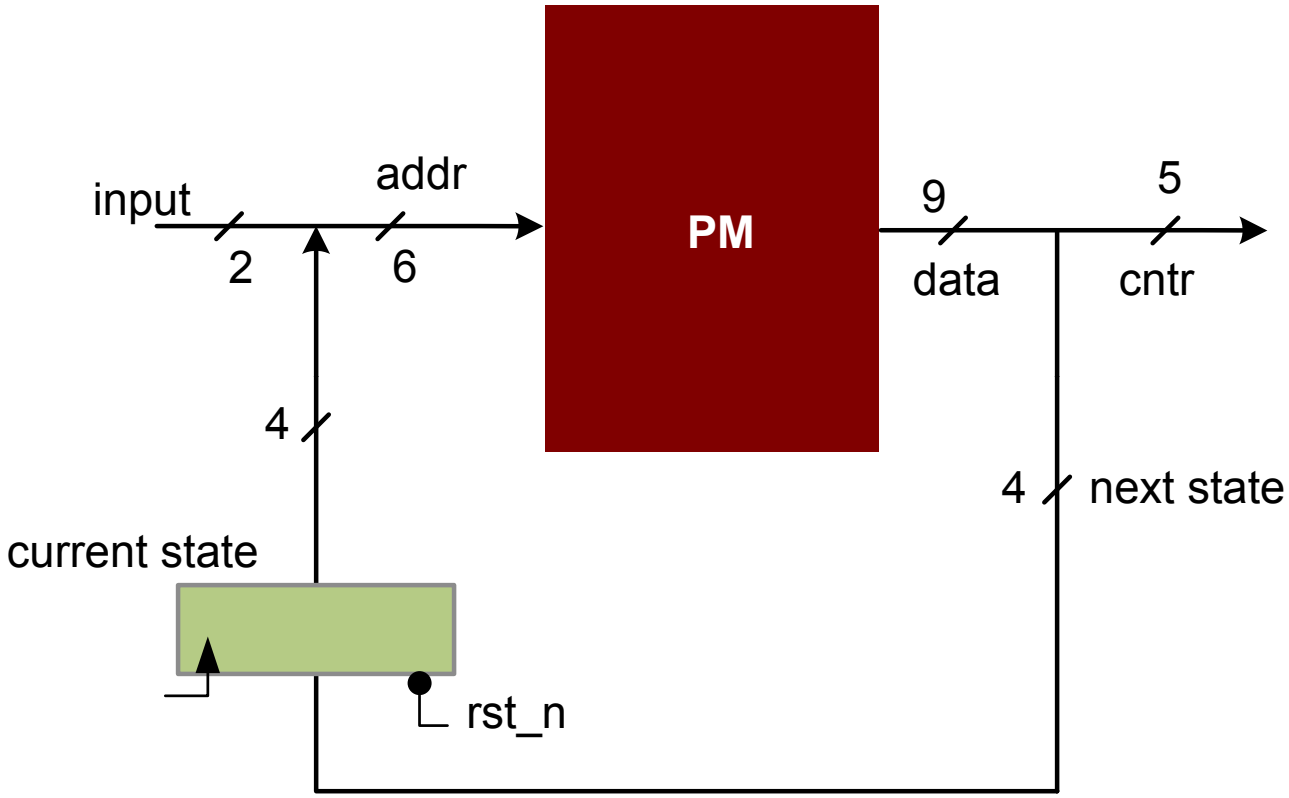
- The control signals for all the states are stored and read from PM instead of being generated by combination logic

PM-BASED MICRO-PROGRAMMABLE STATE MACHINE



- Mealy machine: PM address bits are function of Input and current states, the PM data bits are outputs and next state

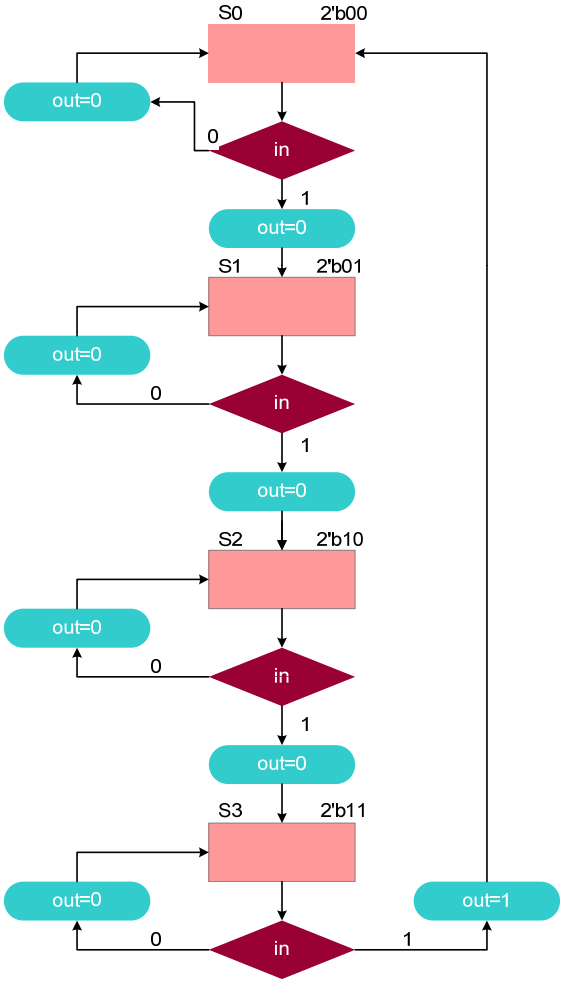
Example Design

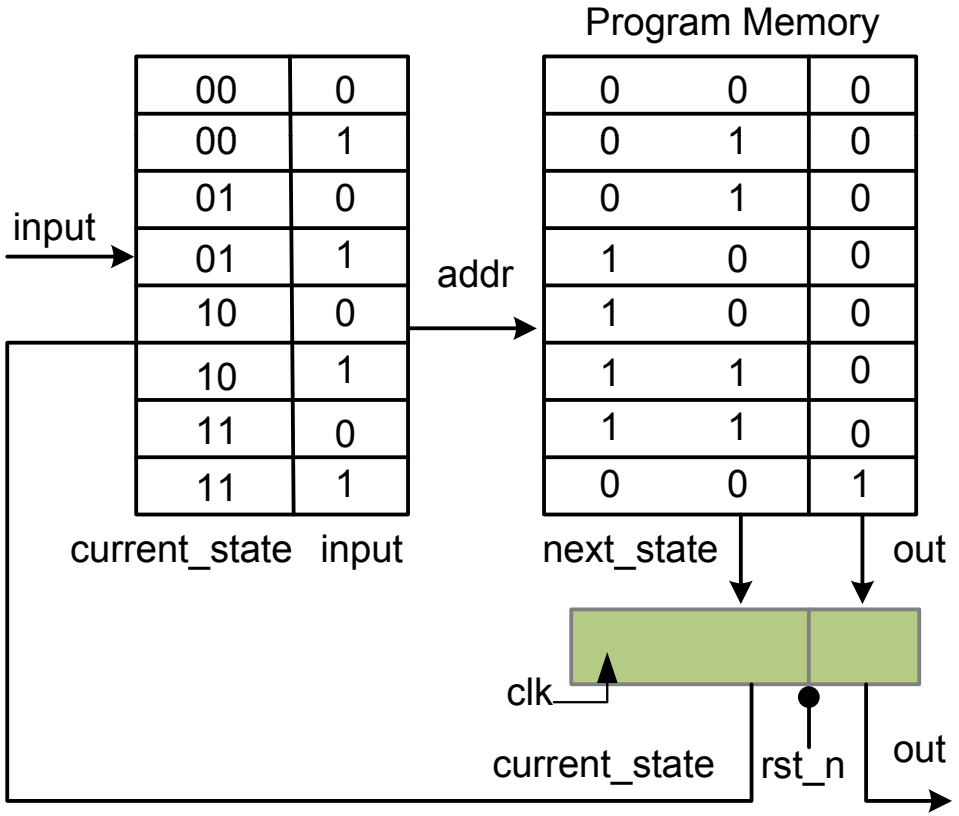


- 2-bit input and 4-bit current state constitutes 6-bit address of PM
- 9-bit wide PM contents are 5-bit output bits for control and 4-bit next state

Micro-programmed Sate Machine Implementation

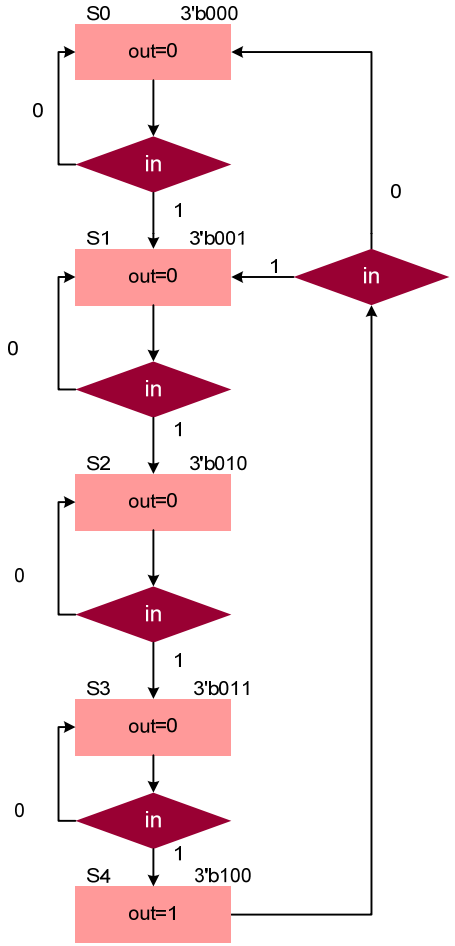
- Number of 1s detected problem of Chapter 9
- Combinational cloud is replaced by PM

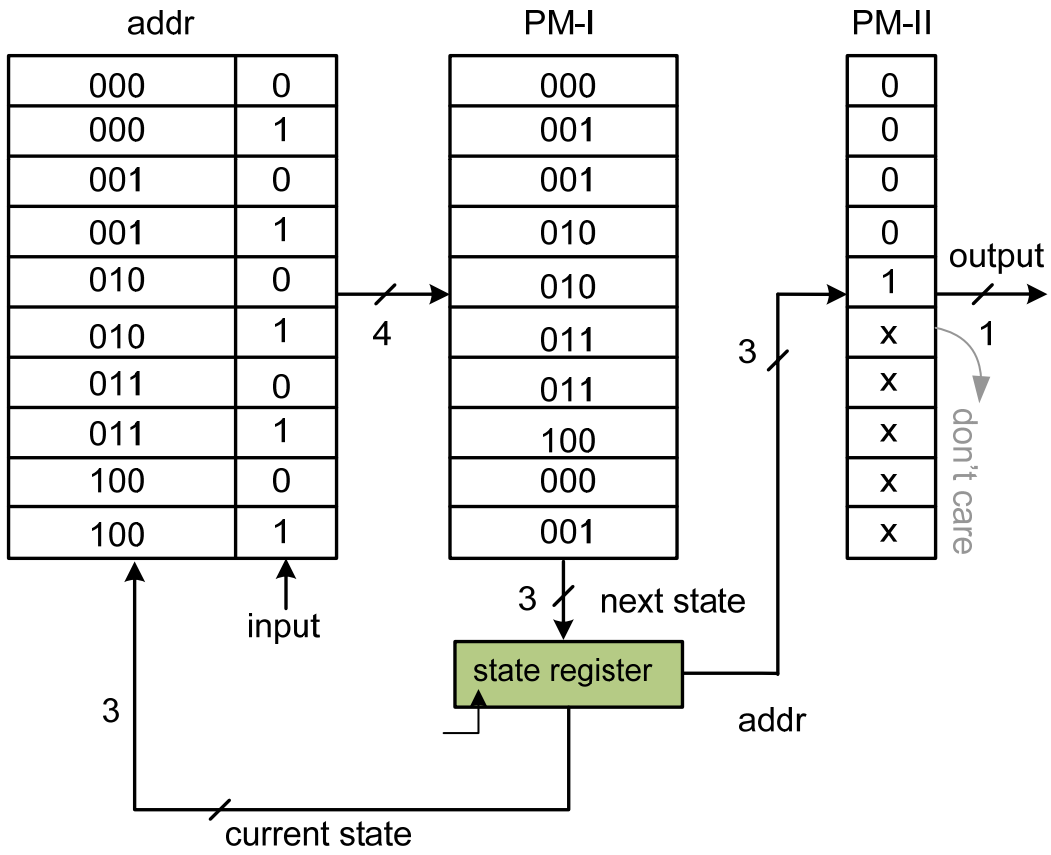




Moore Machine

- Number of 1s detected problem of Chapter 9
- Moore machine two combinational clouds are replaced by two PMs



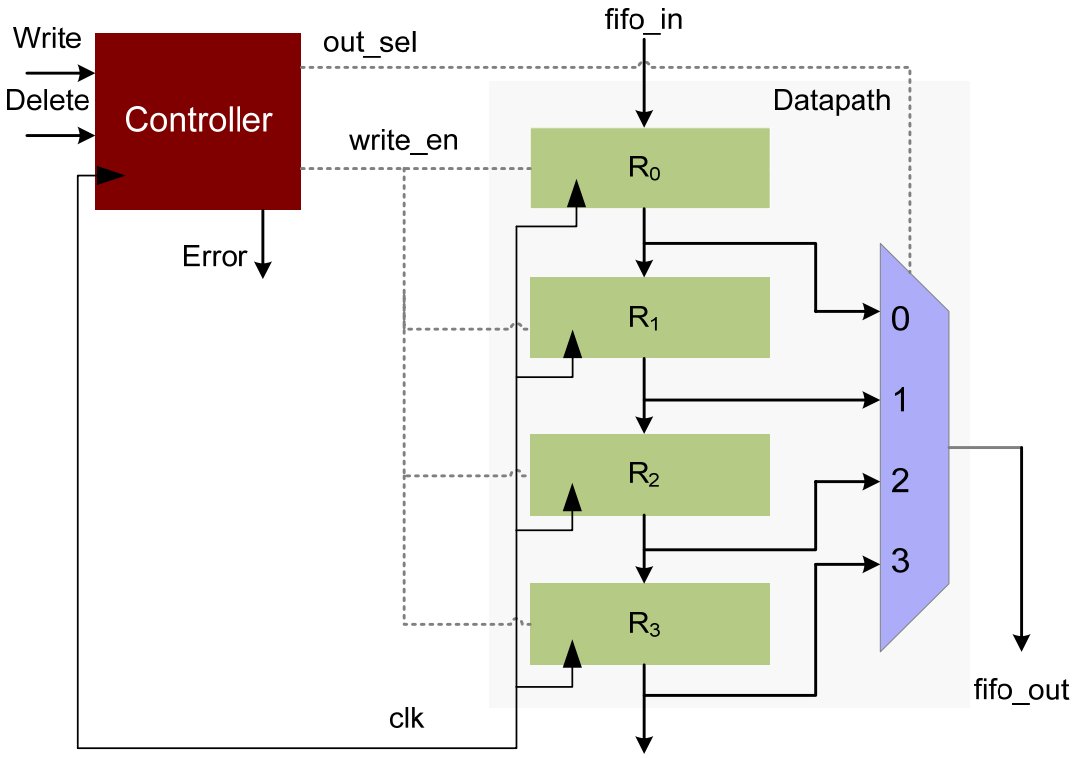


Design Example: 4 Entry FIFO as Micro-program State machine

1. Design a first-in, first-out (FIFO) queue that consists of four registers R0, R1, R2, and R3
2. Write and Delete are the two operations on the queue
3. Write moves data from the fifo_in to R0 that is the tail of the queue
4. Delete deletes the first entry at the head of the queue
5. The head of the queue is available on the fifo_out
6. Writing into a full queue or deletion from an empty queue causes an ERROR condition
7. Assertion of Write and Delete at the same time also causes an ERROR condition

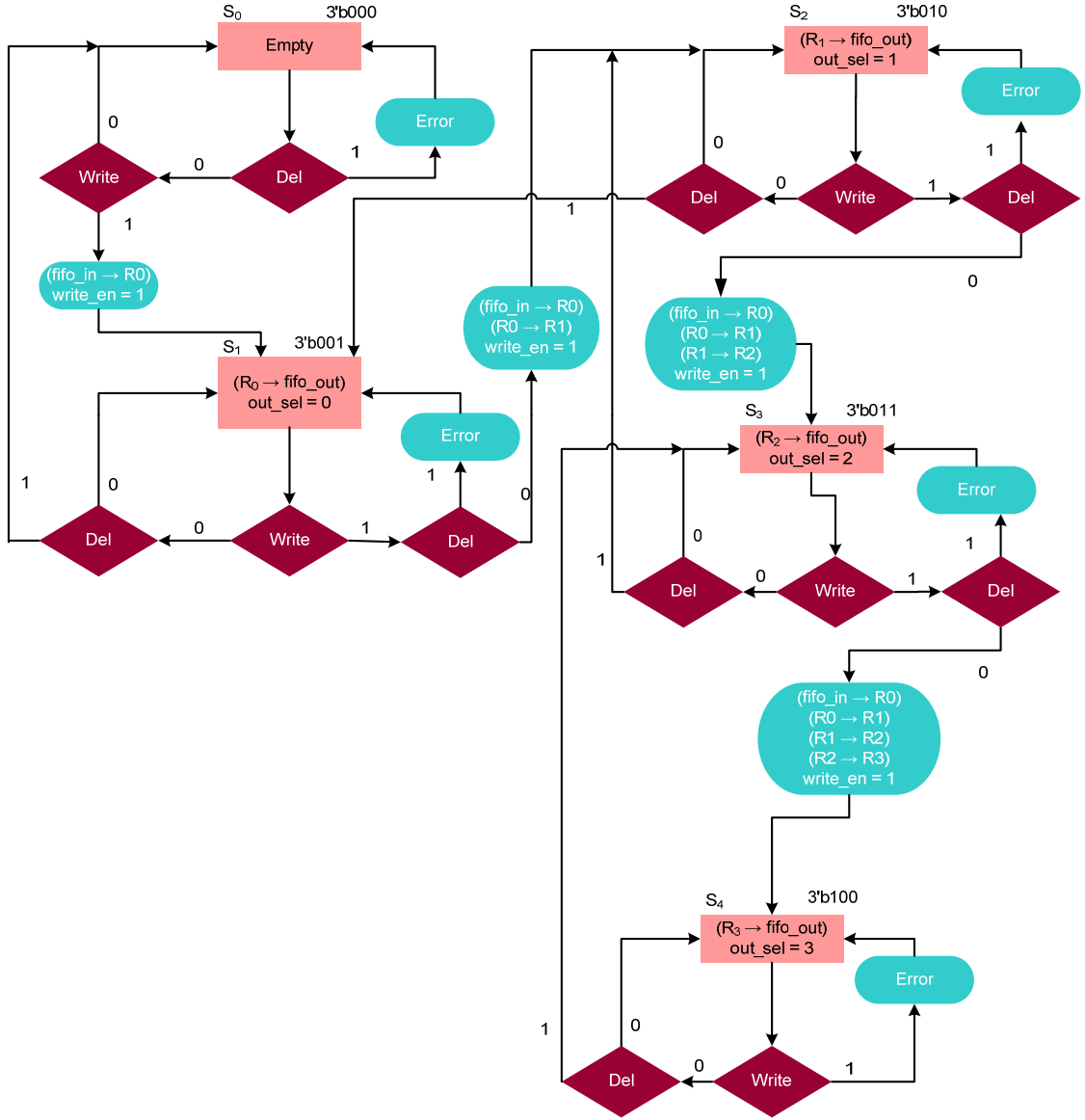
FIFO Design

- Design consists of datapath and controller
- Datapath
  - Four registers
    - Shift registers
  - One 4:1 MUX
- Controller
  - Input signals
    - Write
    - Delete
  - Output signals
    - out_sel
    - write_en
    - Error

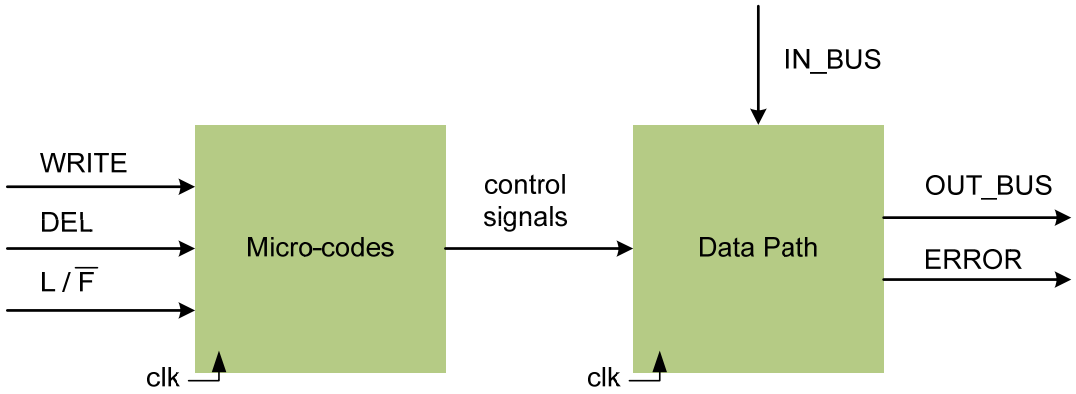


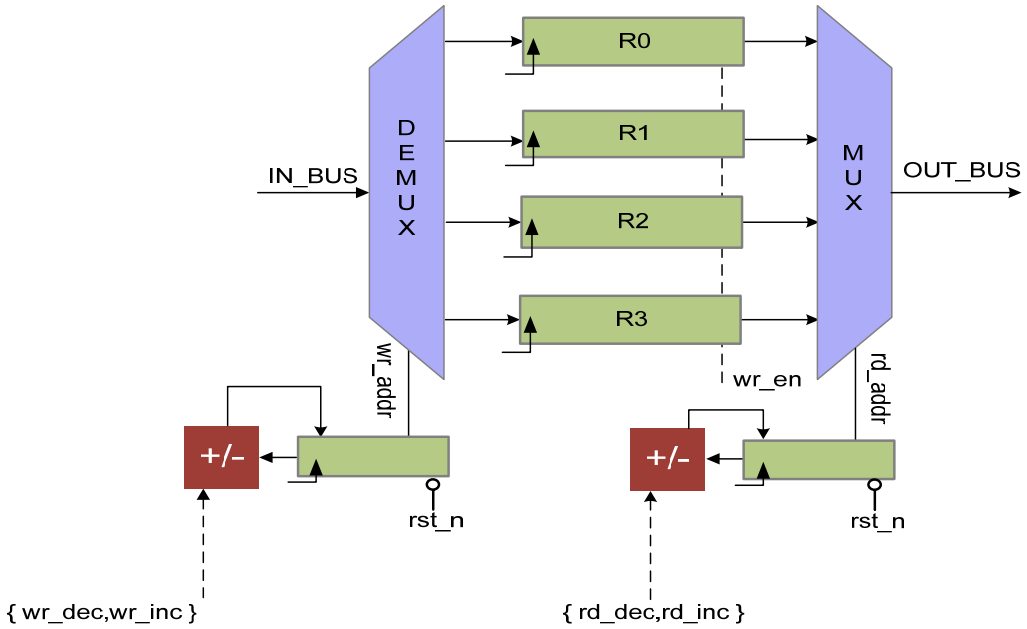
Controller

- FSM based design
  - Mealy machine
- Five states
  - S0: Idle
  - S1: One entry
  - S2: Two entries
  - S3: Three entries
  - S4: Full

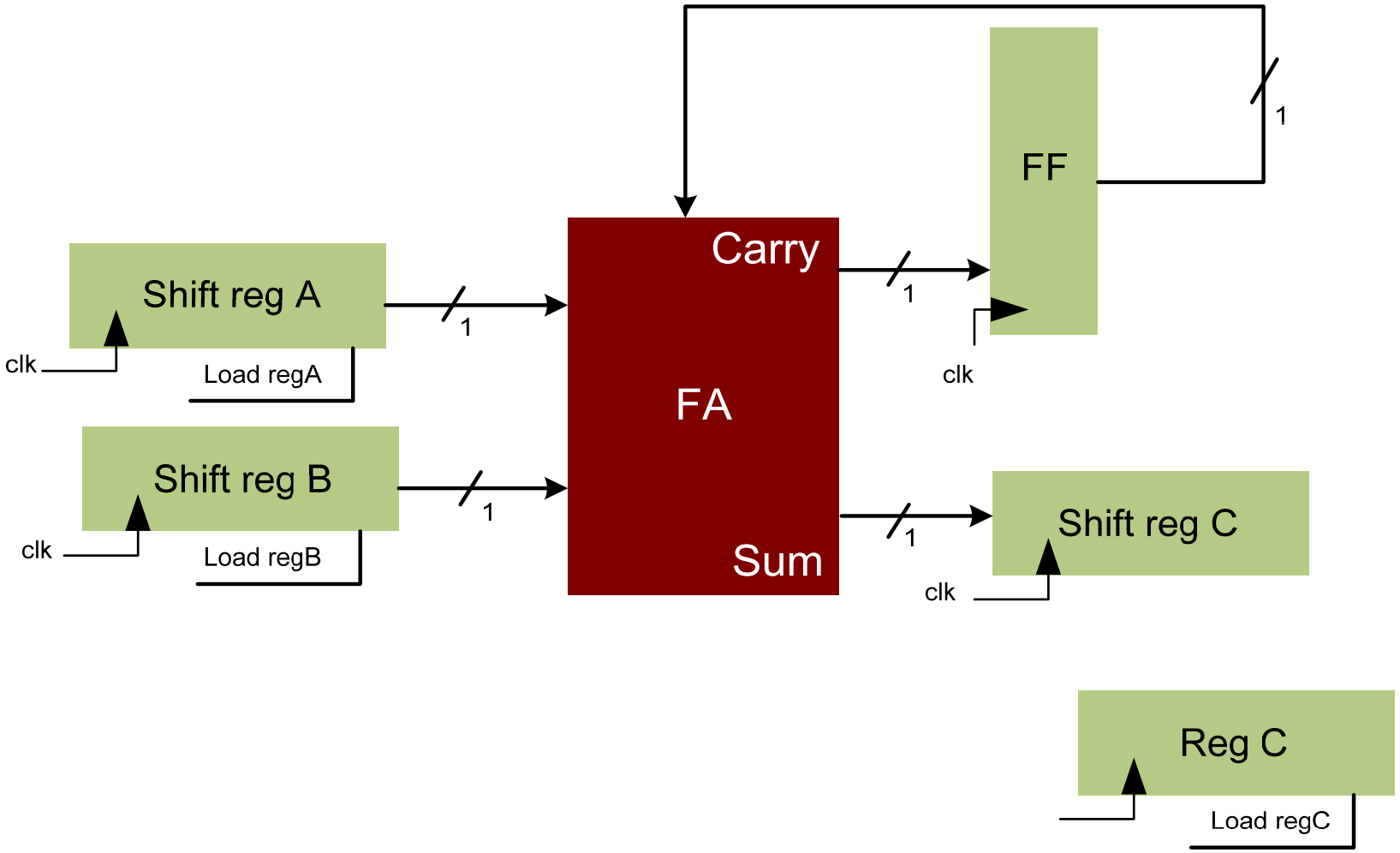


FIFO/LIFO

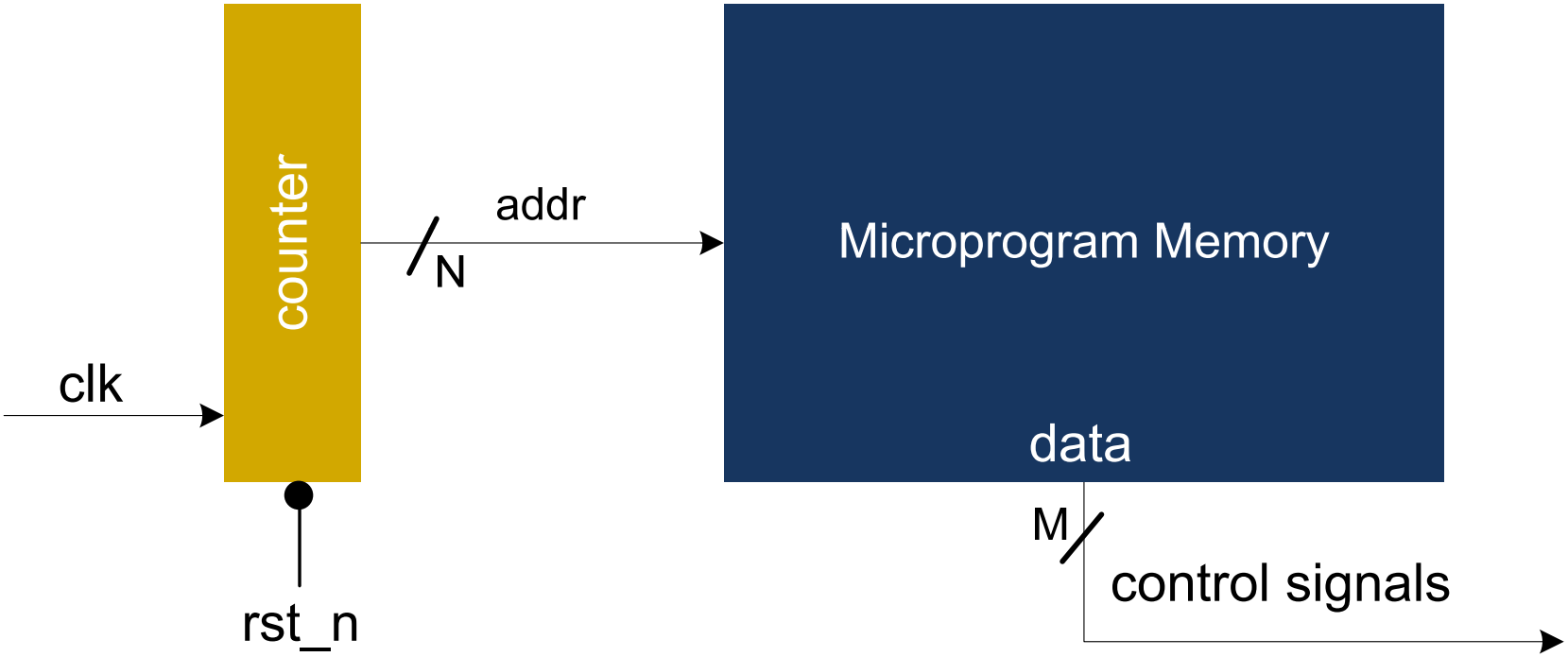

IN_BUS
WRITE
DEL
L / F
Micro-codes
control
signals
Data Path
OUT_BUS
ERROR
clk
clk


IN_BUS
DEMUX
R0
R1
R2
R3
MUX
OUT_BUS
wr_addr
wr_en
rd_addr
+/-
+/-
rst_n
rst_n
{ wr_dec,wr_inc }
{ rd_dec,rd_inc }

Counter-based FSM: Bit serial adder



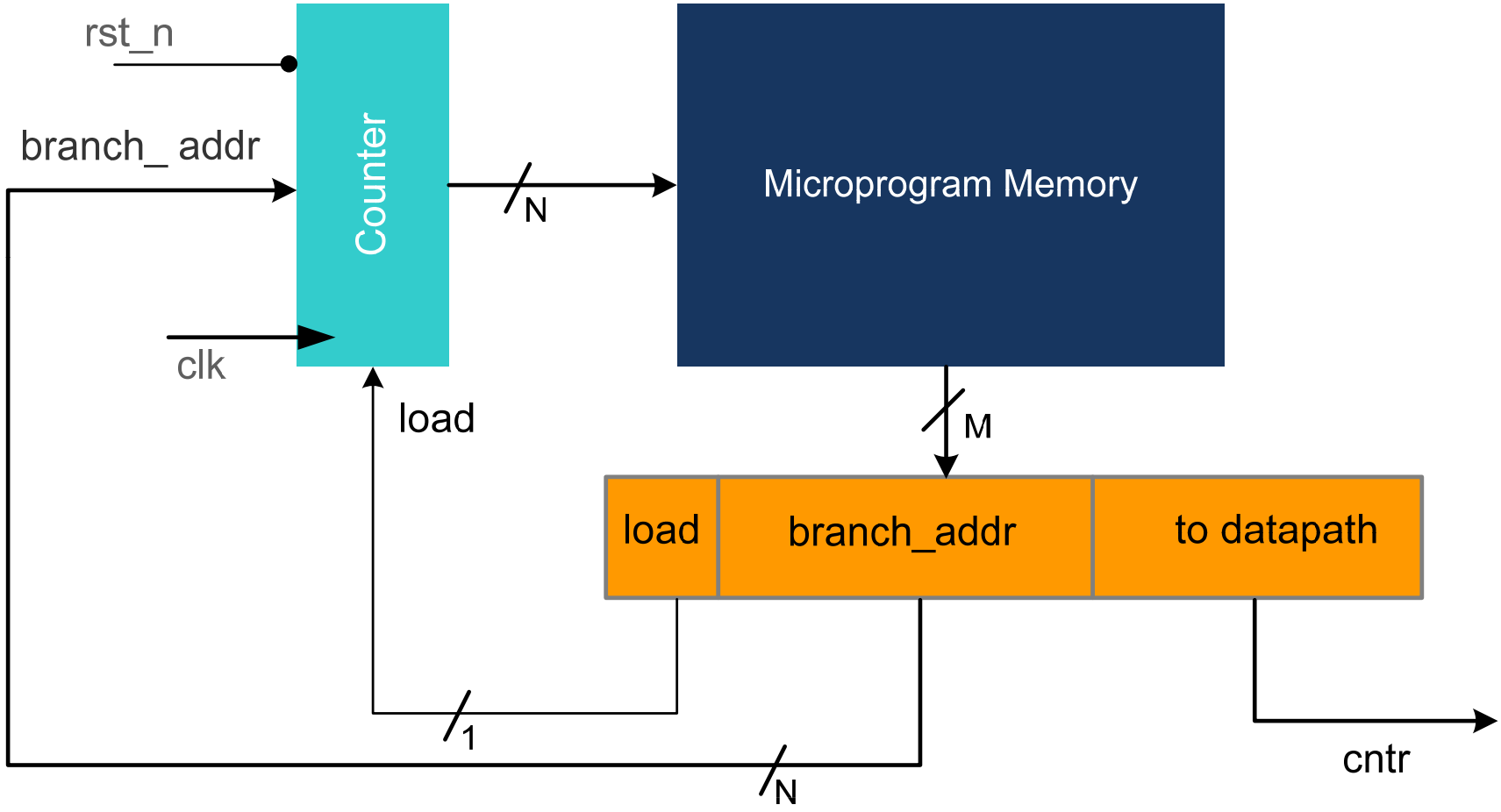
Counter-based State Machine Implementation



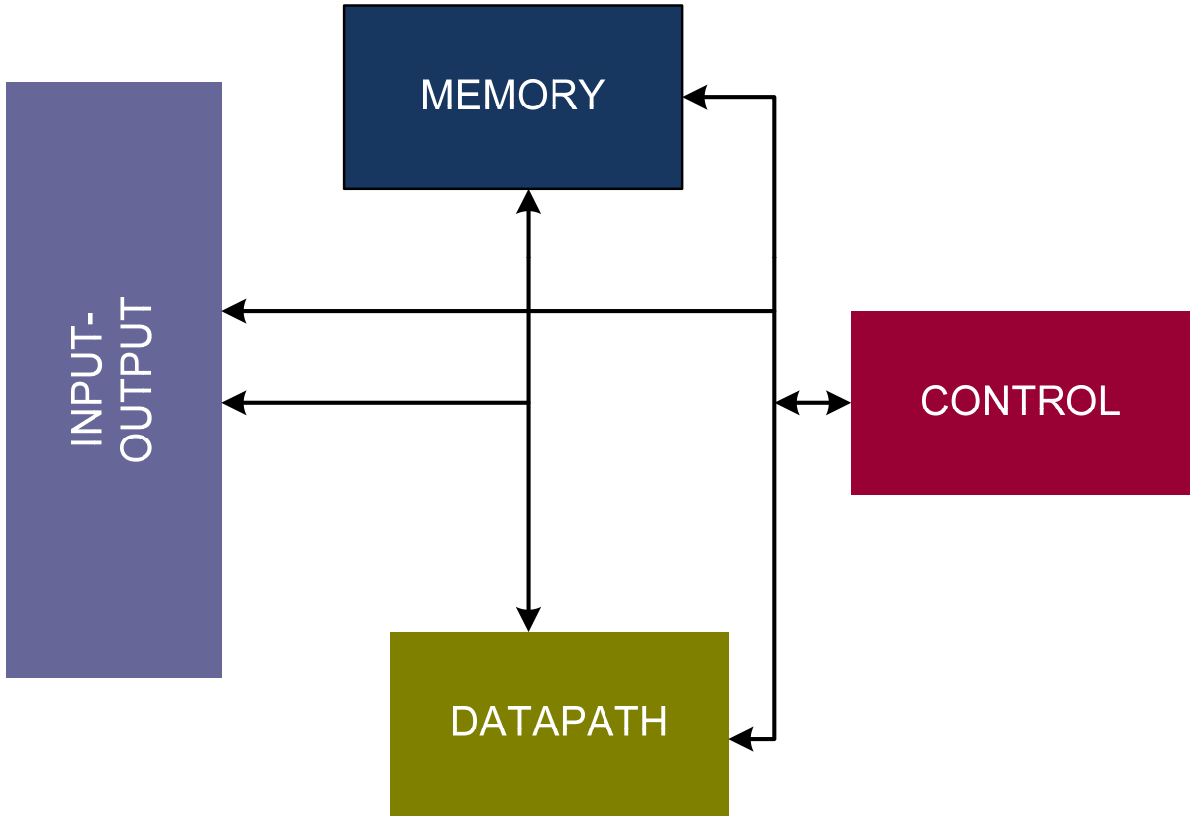
Modifications to Counter-based Microcontrollers 1

- Mechanism for changing count sequence
- Begin another sequence under control of micro-program memory

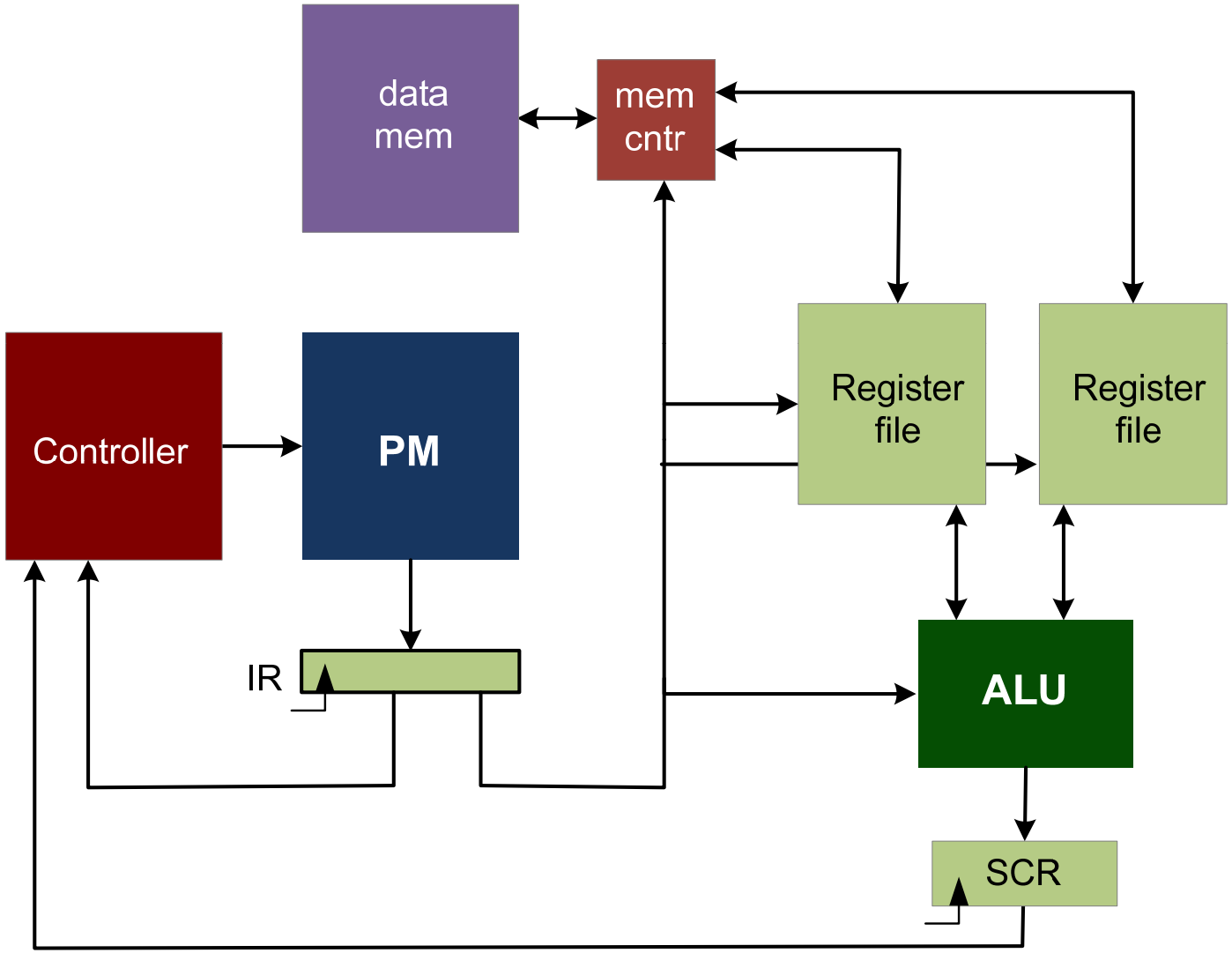
Loadable Counter for *Goto* capability



A Generic Computing Architecture



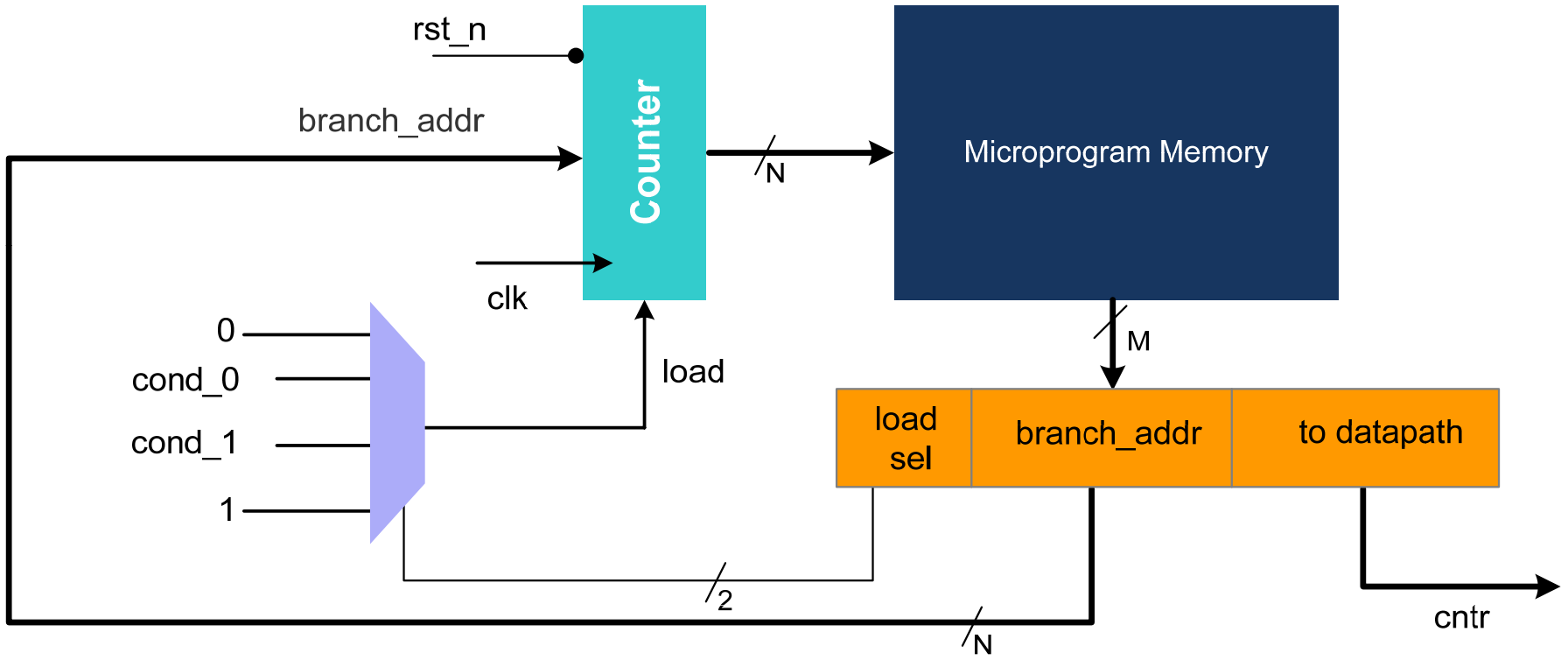
Example: Complete Design



Conditional Execution

- 2-bit control allowing on selection of two conditional inputs and two signals true and false
- The conditions come from the status register in the data path
- Example:
  - zero and positive flags are used for providing conditional execution
  - if (r1 > r2) jump label3; // compute r1-r2 jump if result is positive
  - if (r1 == r2) jump label1; // compute r1-r2, jump if result is zero
  - jump label 2 // unconditional jump

Conditional Branching Capability



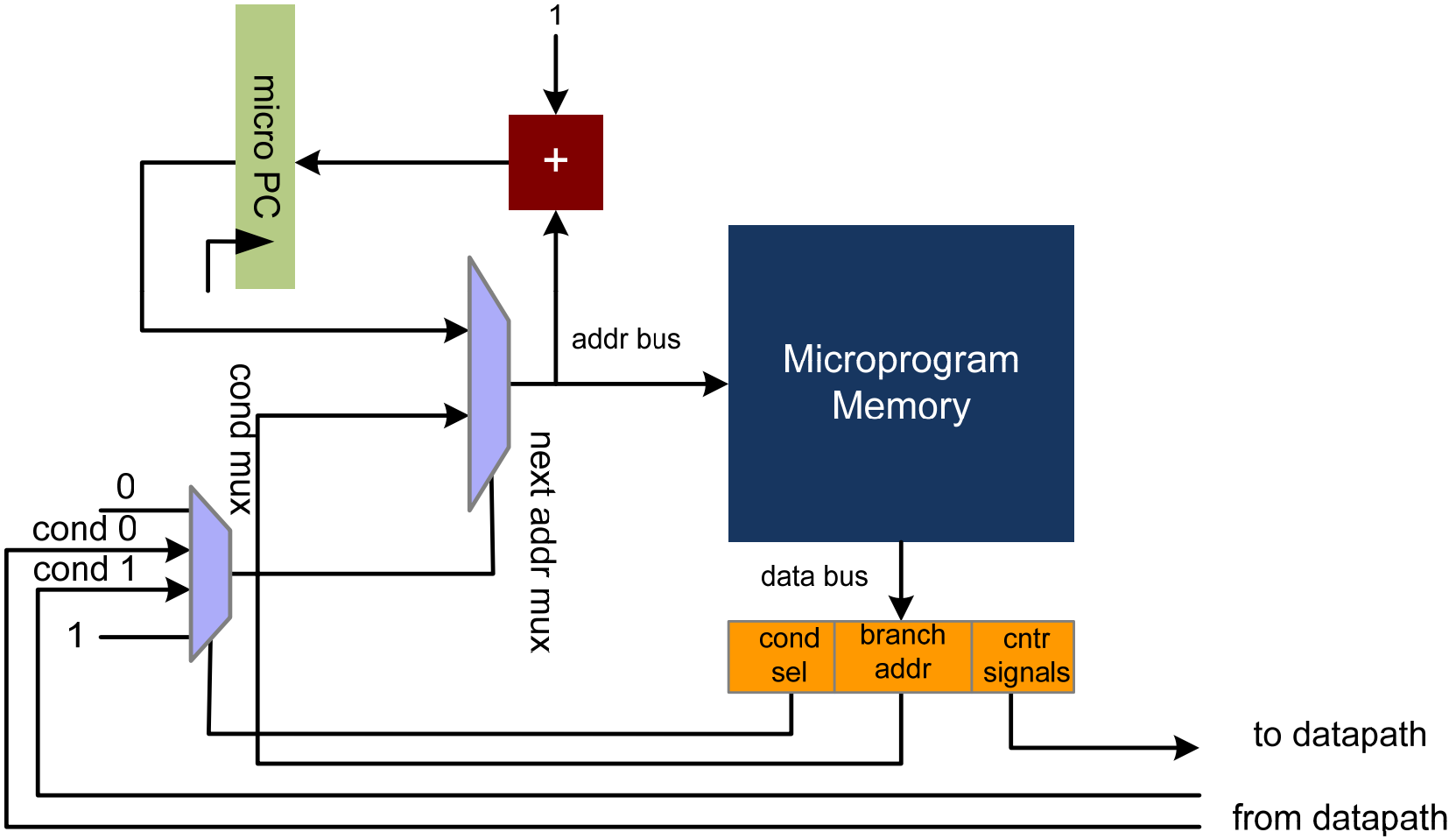
Conditional load

sel	load
00	FALSE (normal execution: never load branch address)
01	COND0 (load branch address if COND0 is TRUE)
10	COND1 (load branch address if COND1 is TRUE)
11	TRUE (unconditional jump: always load branch address)

Pipelined Register

- Often counters are replaced with an ALU based program counter register
- The critical path of the design is long as it goes from the counter to ROM to architecture to functional units generating COND0 and COND1 to conditional MUX.
- The critical path can be broken by inserting a pipeline register in the design

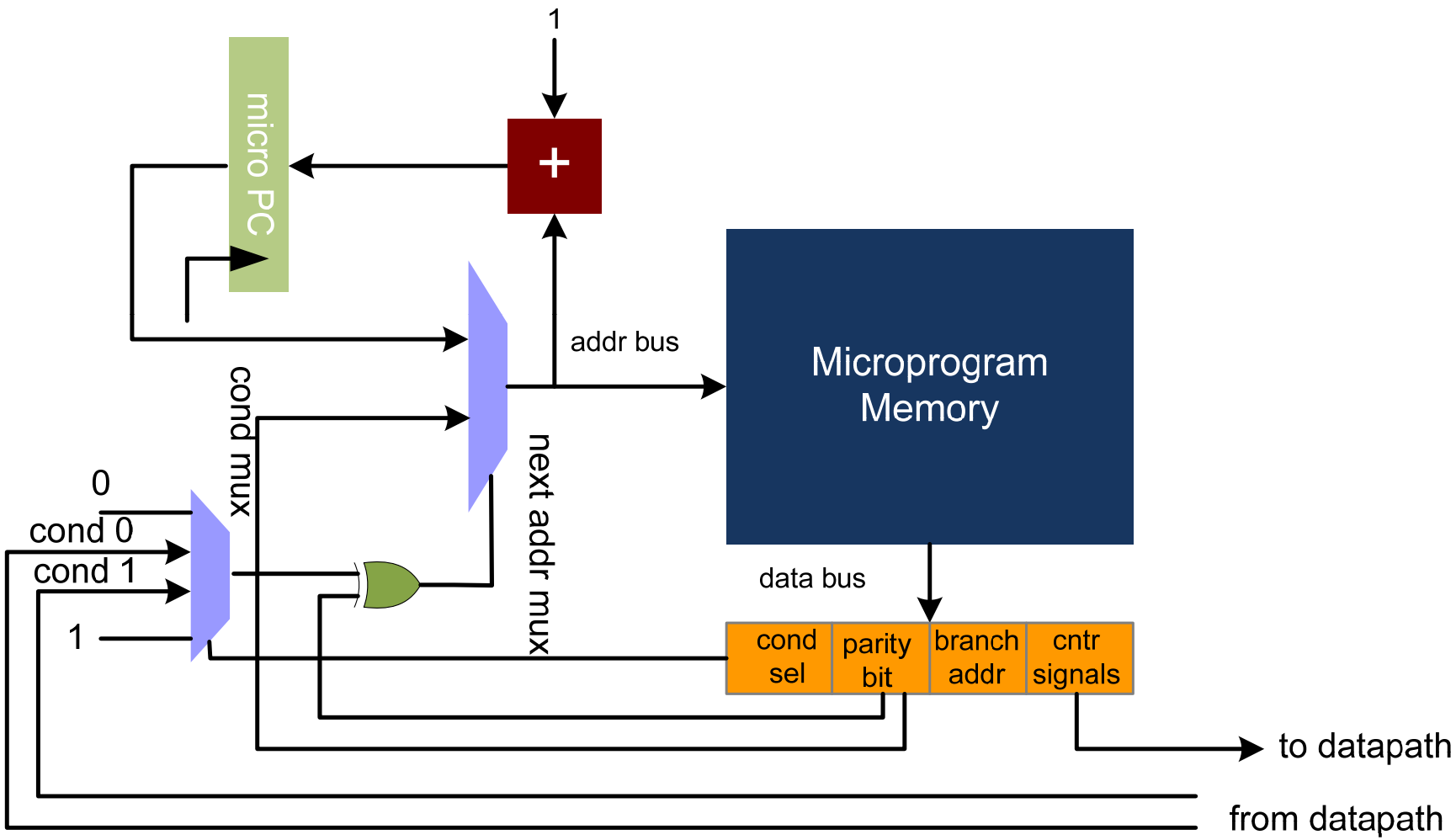
Micro PC Based Controller



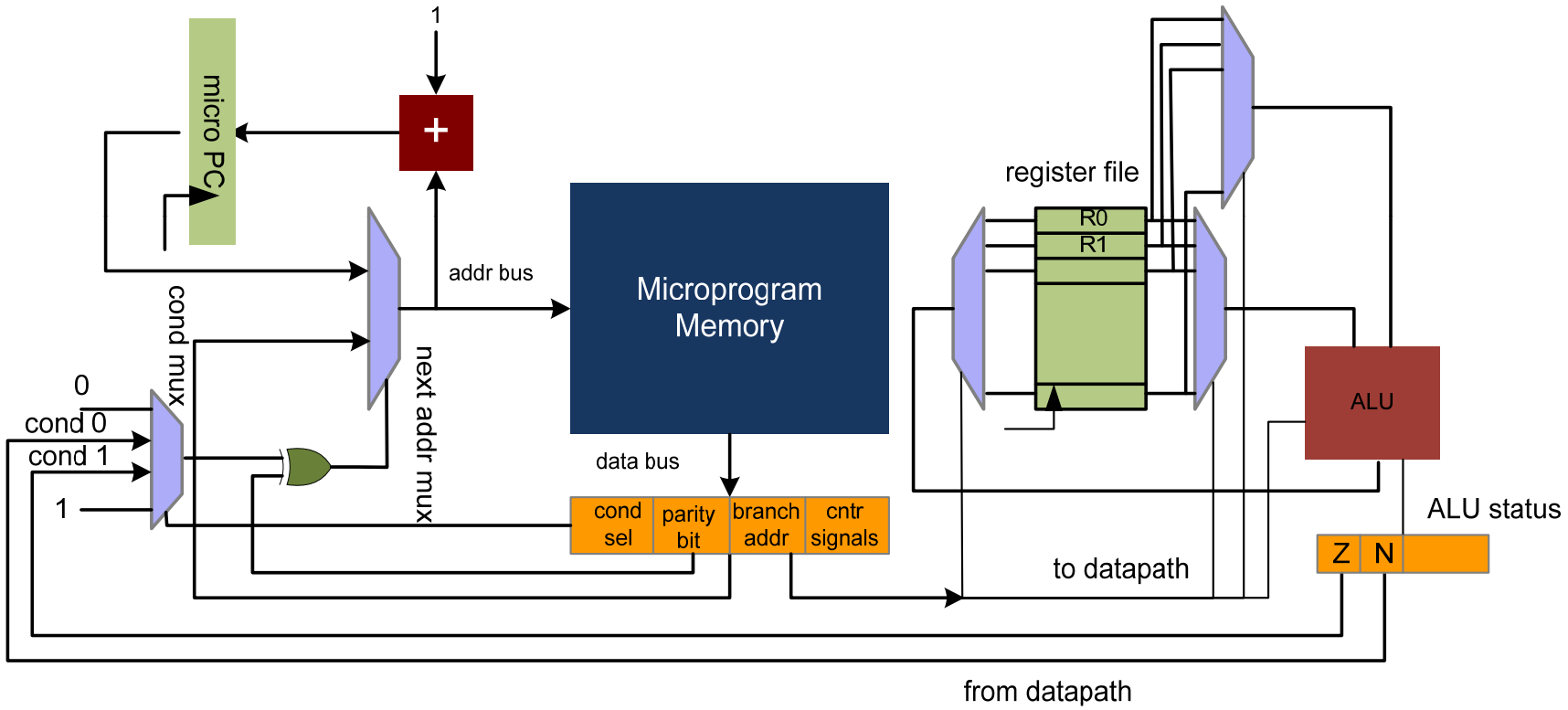
Conditional Branching with Parity

- Conditional execution
  - 2-bit control allowing on selection of two conditional inputs and two signals true and false.
  - A parity bit for inverse selection of the condition
  - Example:
    - zero and positive flags are used for providing conditional execution
    - Inverse selection is provided by parity bit
    - if (r1 > r2) jump label3; // compute r1-r2 jump if result is positive
    - if (r1 < r2) jump label3; // compute r1-r2 jump if result is negative (positive flag 0 and polarity bit is set)
    - if (r1 == r2) jump label1; // compute r1-r2, jump if result is zero
    - if (r1 != r2) jump label1; // compute r1-r2, jump if result is not zero (zero flag is 0 and polarity bit is set)
    - jump label 2 // unconditional jump

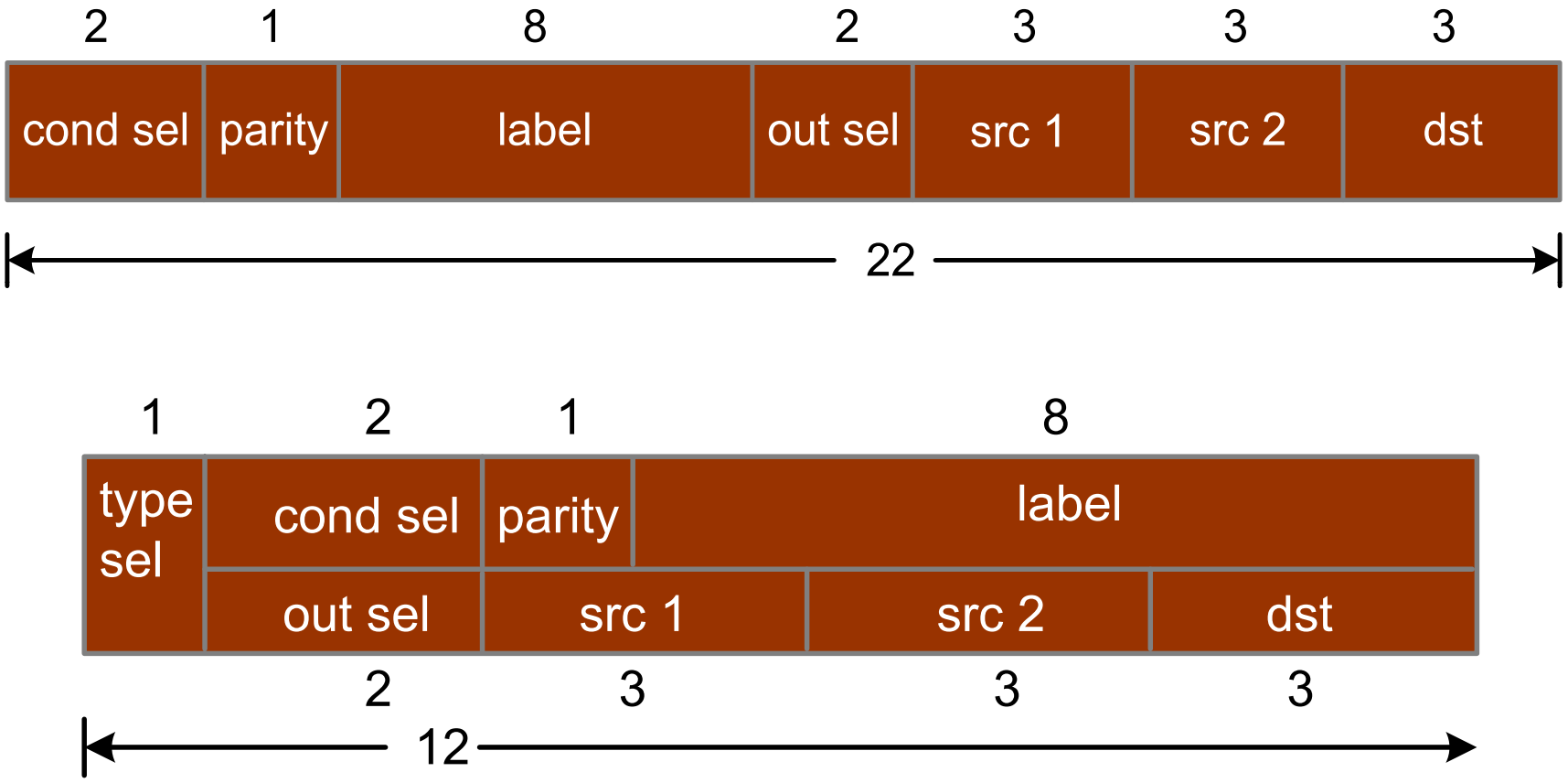
Addition of Parity bit



Controller with data path



Instruction word design



micro PC-based ASM with Conditional Multiplexer